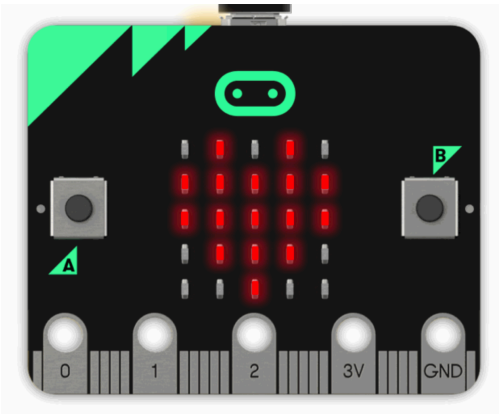
Digital Systems Laboratory Exercises



Hilary Term 2024

Mark van der Wilk — Lecturer & Adaptations
Mike Spivey — Lecturer & Course Designer

Contents

1. Overview 1
1.1. Rules 1
1.2. Instructions 2
Lab 0. Compiling, Running, and Debugging 2
Lab 1. Assembly Programming 2
Lab 2. GPIO and LEDs 2
Lab 3. Serial & Interrupts 2
Lab 4. Operating Systems 2
Lab 5. 2
Lab 6. 2

1. Overview

The course has four-and-a-bit lab exercises, each designed to be done in a single session or less. There is another page with hints for demonstrators.

I've added a couple of exercises related to things in the Trinity Term part of the course. They are there as a way of preserving and making available some existing materials – the Thumb simulator and a simulation of a carry-lookahead adder – in case participants want to experiment with them. There are no lab sessions in Trinity, and no expectation that participants will actually do these labs.

1.1. Rules

The labs for this course will run in a way that is slightly different from other courses, reflecting the wide variety of prior experience possessed by participants.

- No lab reports will be required. However, you may print out your code at the end of each lab session and have it initialled by one of the demonstrators if you want physical evidence of participation.
- In order to get a mark of S for the term, you must attend and participate in the lab sessions. Some participants will want to spend the time in the sessions practising assembly language programming; others will find that easy, and want to pass quickly on to later exercises, or pursue ideas of their

own. All ways of engaging with the material are acceptable, and you won't be assessed on how far you have got through the sequence of suggested activities.
- In order to get a mark of S+ for the term, you must have shown a spark of originality at some point. Please don't ask us to suggest what degree of originality or what idea would be thought of as a spark.
- If you can't come to the lab sessions in person but want credit for them, then you must contact one of the demonstrators to discuss how you can participate remotely.
- We will be giving credit for participating in the lab sessions in a productive way in the context of your previous experience with low-level programming. It will not be possible to get credit for the lab exercises by attending only towards the end of term, whatever completed work you offer to show at that time.

1.2. Instructions

Software resources for the lab exercises are provided via a Mercurial repository, allowing for updates during term. You can browse the repository using a web interface, and instructions for making a local clone are provided as part of Lab zero. There are seven subdirectories in the repository corresponding to Labs zero to six, plus a setup directory used in the installation process. Several files appear as independent copies in multiple directories, for example, the file hardware.h that specifies the locations of various I/O registers on the micro:bit, and the file startup.c containing the code that runs when the micro:bit starts up. As far as possible, all files with the same name are identical, but in any event, they are consistent with each other. Naturally enough, each directory has its own unique Makefile.

Other pages contain information about programming the micro:bit and using the Linux-hosted toolchain to compile programs.

- A **very quick quide** to programming in C.
- Links to hardware documentation.
- Instructions for installing the toolchain on your own machine.
- Step-by-step instructions for capturing micro:bit signals with an oscilloscope or logic analyser.
- A note about controlling the display on the micro:bit.

Hints for demonstrators.

Lab 0. Compiling, Running, and Debugging

Lab 1. Assembly Programming

Lab 2. GPIO and LEDs

Lab 3. Serial & Interrupts

Lab 4. Operating Systems

Lab 5.

Lab 6.